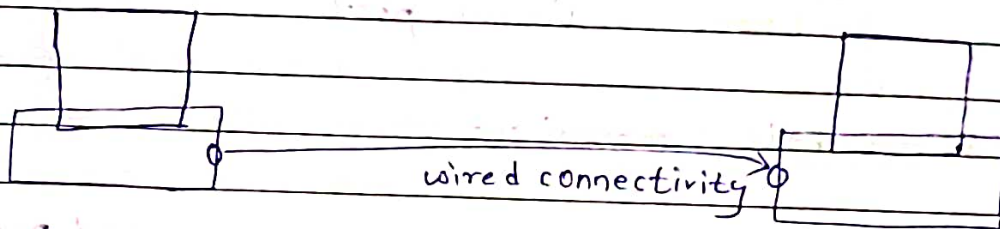
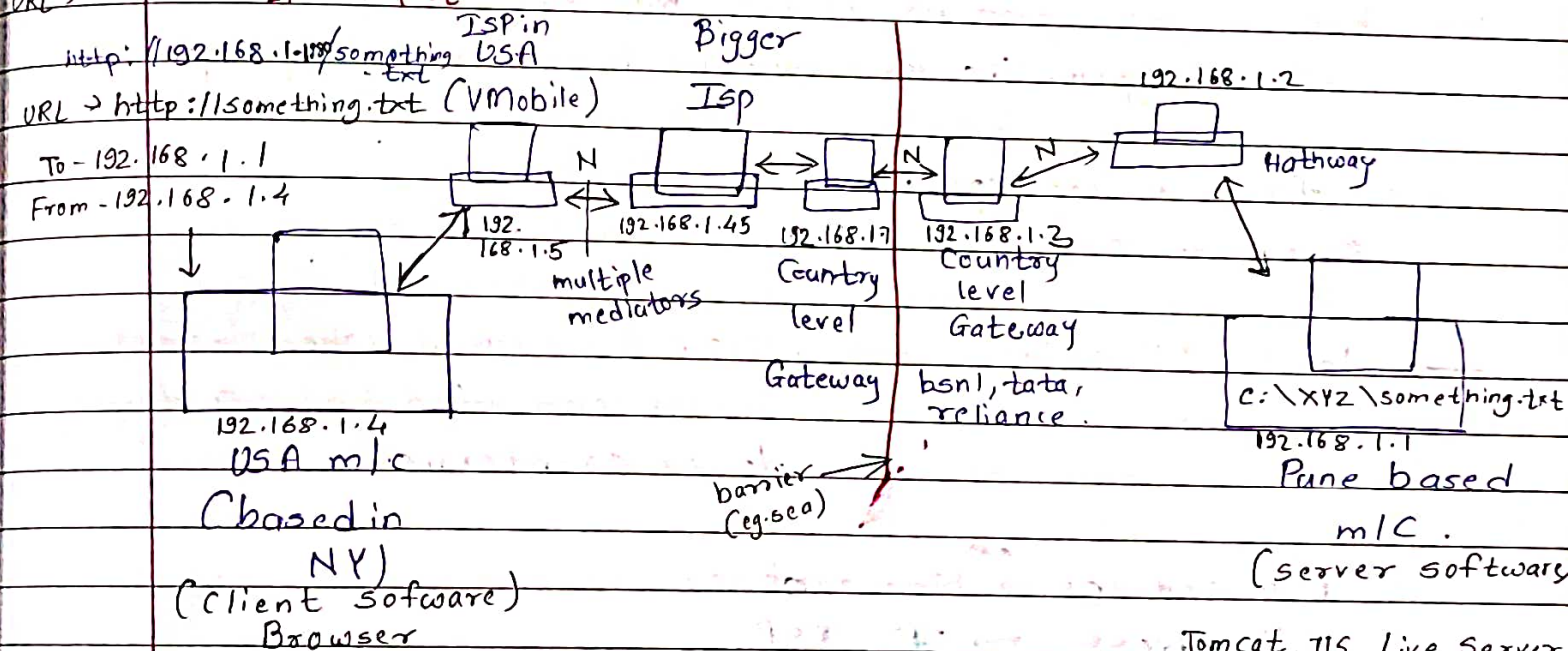


Day 1 Web Programming Technology (WPT)



URL → $http://192.168.1.1:80/something.txt$
 IP port
 port no. of server is req. client not req becoz it will take random port.



IP address is provided by ISP.

Browser software → software that do networking job. handles communication betⁿ two m/c far apart.

Server software →

standard

http - hyper text transfer protocol → decides size of packet.

contains info of packet.

means text & images.

→ port → through which something can come in m/c or go outside. - to send/receive packets to m/c/software
input port, output port,
soft ports → virtual ports

port 80 → http communication

→ port no. is ~~not~~ fixed when particular soft/program is connected to diff. m/c.

eg MySQL → reserves port no 3306.

→ but in java port no. is not reserved. becoz it was client program. like for random, browser based program.

facebook uses apache.

Some software reserves port some not.

→ local IP address → 127.0.0.0. → used when internet is not connected but u r trying to use as internet.

Mac address → ^{mn}Manufacturer

IP address → ISP.

need to have same IP throughout year.

Static IP → for server software.

Domain name sellers

(broadband).

Agents: eg. GoDaddy, avoids conflicts.
.com, .org.

we have to renew domain name after some years.

Domain registrar → uniqueness check.

to check IP of something.com:
unnoticed

something.com → 192.168.1.1

↓
ping back request

↓
mapping done by

Domain Name Services (DNS)

ISP have DNS ka mapping, In general, ISPs are vendors request goes to IP when we type .com on browser not directly to server.

Uniqueness of IP → maintained by ISP.

Day 2.

Server:-

Converts required files into bits & bytes data privacy.

encoding

receives request & find out ^{that} file at ^{particular} ~~that~~ location.
request to address mapping is done by server.

When we send req using http, browser uses a random port to send a request, server used particular port for commⁿ.

ping of death → to get domain name from ISP contains some malicious/virus program inside it & this is handled by firewall.

Wireless → UDP protocol.

Wired → http ... behind the scene TCP is used.

In wired, acknowledgement is received on browser end when all packets are received at server end. browser waits for acknowledgement if not received it resends particular packet.

- Firewall:- some rules setup for communication
- only allows or process packets which follow protocols.
 - for ex. if a site is banned & we send request for that site then request is destroyed at firewall itself.

- Proxy Server:- mediator server, firewall is installed at that. can manage the traffic

- router → like switch case → have to take decision multiple
- Switch → if else → only 1 decision, True/False

switch is not more intelligent device router is most intelligent. If u have 3 ISP connⁿ, BSNL, Jio. router can choose who's bandwidth is ↑ & which to use at that time.

Chopping of packet & numbering is done in OSI layer.

code. → to open VS code through cmd.

live server → coupled with VS code. can't run w/o VS code hence it's a development software.

IP address : Port no / filename.

127.0.0.1 : 5500 / something.txt

Ctrl F5 → hard refresh / forceful refresh.
only F5 → shows cache copy.

F-12 → Network → Ctrl+refresh → headers → ① General

② Response header

③ Request header

* Hypertext markup language :-

html parser → runs at backend.

based on content type parser is called.

it parses the file → acc. to given instrⁿ.

Right click → view source

* World wide consortium → give instrⁿ related tag
tag name → process, how it works.

vendors have to implement the program for parsing...

→ they have not kept upper bound

<head

</head>

<body>

body
↓

break
↓

 something else
 something else

 abcd
 abcd

<h1> Hello </h1> Hello

<h2> Hello </h2> Hello

<h3> Hello </h3> Hello.

</body>

h1 - h6 - special tag 6 diff. font size, takes implicit break internally.

hence no need for br in h1-h6.

<hr> → horizontal ruler - prints a horizontal line.

`<p>` `</p>` - prints the paragraph betⁿ & prints a blank line.

`<title>` -- Name of tab, topmost bar.

`<img src = "logo.jpg" alt = "Sunbeam logo" height = "200px" width = "200px"`

1 pt on screen & 1 pixel

`</mahesh>` → fall back mechanism

there is no mahesh tag still html still is giving o/p not error.

here it pres. prints contents inside it

ex `<mahesh> aise hi </mahesh>` → will not give error. prints aise hi

- Object tag - Introduced by microsoft - req plugin & plugin Blink - Firefox are not safe to download/install

www.schools.com → html → tut.
mdn web docs

Day 3.

1) `html:5` tab → gives structure of html (basic)

2) center → To move text to horizontal mid

`<center> Hi </center>`

- not allowed now - deprecated so vs code show it in red colour but browser will use

- 3] `<!DOCTYPE html>` → To convey & ask browser to use html5
 - Every browser understands html5 ^{browser} version
 - it will affect while working with crossbrowser

- 7] `<!-- -->` → ctrl+k+c - to comment
 → ctrl+k+u → uncomment

- 5] `<a href="..."` ^{next webpage path} To navigate to other webpage.
 click to go to 2nd page. → anchor tab

`</a>` → This open in same tab window

- 6] `<a target = "_blank"` → target tells to start
`href = "https://adac.in">` new site on new page
`<img src = ...>`

styling → Sample 6

`<p style = "color: yellow; background color: brown;">`
 This is text
`</p>`

`<table>` `</table>` → to create table
 table row → `<tr>` `</tr>` → for rows
`<td>` `</td>` → columns.
 like a container table division.

} sample 7.htm
 Table.

→ colspan → To merge columns - including the current column.

→ rowspan → merge rows vertically. page sample 8

• sample 9 - table in table

margin, padding, border.

user-agent - Stylesheet → Styles → colour, cursor, text-decoration.

input → to take ip from user

<input type = "text"> ← default text form

Day 04

1. Form Tag

Form methods - GET/POST

Style → inline vs External vs Internal

* Form Tag:-

<input Type = "Submit" value = "Click me" >

↑
shows this text when we want to click button.

<Form action = "MYPROGRAM.JSP" >

Name: <input type = "text" >

Address: <input type = "text" >

input Type = "Submit" value = "click me" >

By default method type is GET → data sent in insecured manner.
 method decides type of data like data will
 On the packet or inside the packet

here we have not connected to java so when we press submit button it gives 404 error.

when we click submit MyProgram.JSP is called.

When we send data on POST - comparatively secured

It can be sent in Payload in inspecting (F12)
near header

When we press submit button, request is sent to JSP pages.

* required :- used to validate:

if used for name to be entered: can't be kept blank

also used to give range for age (18-60).

used for email & password:

$\uparrow [a-z][A-Z][0-9] \cdot [a-z]@ [a-z] \cdot com \downarrow$
start of reg. expⁿ regular expression end of reg. expⁿ

Consistency is not there in tags of Datatypes -

No consistency in case of ~~html~~ html 5 features.

- validation no. so we use javascript for validations

Type = button: $\rightarrow$ javascript frequently used
 $\rightarrow$ ask for javascript.

Type = submit $\rightarrow$ packs data & sends itself.
 $\rightarrow$ provided form tag is there.

~~type = text~~ type = "text", maxlength = 10

placeholder, required, maxlength
javascript → browser side execut".

It is compulsory to write attribute Name becaz it's a key. If we don't write Name data is lost. compulsory when we use form & submit.

* DIV Tag:-

- Div is a lookless container. can accept anything
- Block element
- occupies entire width by default.
- block control → becaz another div block is placed after current block.

ex: Hi
 Hello

border-width : 2px shorthand

border-color : red

border-style : solid

→ border : 2px solid red
 syntax of border here sequence doesn't matter.

* Styleclass -

Internal Style → <style> (applicable within that file only)
 when we write we have to write class = MyStyle every time.
 classname → MyStyle {
 background-color : yellow;
 border : 2px solid red;
 }
<style>

<div class = "MyStyle">
 Div1
</div>


```
<div class = "MyStyle" style = "background-color: green;">
```

```
div {
  display: block;
}
```

local style is always preferred.

TestStyle2.

```
<div class = "MyStyle YourStyle">
```

multiple class call. / ↳ here whichever class is written first that class is called first.
This sequence doesn't matter...

CSS Cascading ~~Simple sheet~~ → style sheet.

```
<link rel = "stylesheet" href = "allstyles.css">
```

JSP to html & JSP to CSS is conversion in background.

external

here no dot(.)
so we don't have to specify class name

```
div {
  color: white;
}
```

→ here div is inherited implicitly

External style - When we use css file.

applicable in any file

Inline :- When style is written inside div block itself.

validation, password check, animation → javascript

Day 5

JavaScript → Test JS01 → for validation, animation

JavaScript engine → executes javascript program.

HTML parser → executes html program.

script tag can be written in head & body.
but to write in body is preferred.

<script> — </script>

<script>

var v = 100; → here v becomes int

v = "abcd"; → here v becomes String

v = true;

v = undefined; → here v is undefined. ^{special value}

</script> v = null; → v null means v is object

Number, Boolean, String, Undefined, Object.
↳ data types in JS.

No further categories in number like float, long.

console.log (typeof (v));

↳ To
print
on console

↳ gives type (data type) of v

* Type conversion:- implicitly converts into bigger data type

ex. we use '+' op. betⁿ number & string ans will
concatenation of num & string.

1) `var x = 100;`
`var y = true` $\Rightarrow 101 \Rightarrow 100 + (1)$
`var z = x + y;` $\hookrightarrow \text{true}$
`console.log(z)` hence 101

2) `console.log(parseInt("100"))` $\rightarrow 100$
`console.log(parseInt(true))` $\rightarrow \text{NaN}$ - not a number
don't convert boolean to int
`parseInt(null)` $\rightarrow \text{NaN}$
`parseInt(undefined)` $\rightarrow \text{NaN}$

this means parseInt work only with string & if that string contain number only.

$\hookrightarrow$ `parseInt("abcd")` $\rightarrow \text{NaN}$ - string but no digit so NaN.

```
* var v = undefined
  if (v > 100)
  { console.log(v + " > than 100"); }
  else
  { console.log(v + " < than 100"); }
```

here v is undefined - data type
`v > 100` $\rightarrow$ it can't compare undefined with number hence gives false

$\Rightarrow$ if (`typeof(v) === "number"`)
always check the type of variable before proceeding otherwise it gives unexpected output.

→ Debugger:- asking javascript engine to stop for a while then resume when developer says

Q. Why script written in body?
if code is dependent on html controls ie (to take input from user) write JS in body & towards end of body
reason becoz of dependency on UI elements or debugger

< head >

: < title >

< script >

↓ html parser

↑ Java Script engine

(starts from script)

F10 - debugging → watch - progress in debug sources

* Functional Programming in JS :-

Every JS fn returns something

① If u have return statement it will return that
② but if no return statement, it returns
undefined.

1)

function Hi()

{

}

var result = Hi();

console.log(result); → undefined - ②


```

2) function Add(x,y)
    { return x+y;
    }

var result = Add(10,11)
console.log(result); → 21

```

Add(10, true) → 11 10+1

Add(10, null) → 10

Add(10, undefined) → NaN → Not a number
~~console.log~~

```

3) var Add = function(x,y)
    { return x+y; }

console.log(typeof(Add)); → function

```

Function name → variable → case sensitive

By default type of parameter is var.

There is no type of parameter, declaring a parameter do not require parameter.

ex. add(x,y) { } ← here data type of x,y is not written it captures implicitly type

```

→ { var f1 = GetMeSomethingToExecute(11);
    f1();
    → GetMeSomethingToExecute(11)();

```

Test JS 04

Page No.

Date

Passing "fu" as a parameter → more frequently used.

```
var Add = (x,y) → { return x+y; }
```

```
var result = Add(10,20);  
console.log(result);
```

```
var Add = function (x,y) {  
    { return x+y; }  
};
```

instead of this use

* Literal Syntax for class? - not used generally.

```
var person1 = {  
    Name = "Mahesh",  
    Age = 42,  
    Print: function ()  
    {  
        console.log(this.Name + this.Age);  
    }  
};
```

```
console.log(typeof(person1));  
person1.Name = "Pooja";  
person1.Print();
```

→ mahesh 42

* Class using Constructor Function :- M. imp. used here to declare class "fu" is written frequently

```
var Person = function (name, address, age) {
```

Name, Address, Age class members

```
    this.Name = name;  
    this.Address = address;  
    this.Age = age;
```



```
-this.Print = function()
```

```
{
  console.log(this.Name + this.Address
              + this.Age);
}
```

```
{
  var p1 = new Person("mahesh", "pure", "42");
  var p2 = new Person("Pooja", "jalgaon", "60");
  p1.Print();
  p2.Print();
}
```

here print is an object (var) that stores function.

* Class writing using Class (ie. constructor)

Syntax : ES6 :-

Class Person

```
{
  constructor (name, address, age)
  {
    this.Name = name;
    this.Address = address;
    this.Age = age;
  }
  Print ()
  {
    console.log (this.Name + this.Address + this.Age);
  }
}

var p1 = new Person ("Pooja", "Jalgaon", "24");
var p2 = new Person ("Nilesh", "Pune", "43");
p1.Print();
p2.Print();
```

```
var x = 100
var y = 200
var z = x + y;
console.log(z) → 300
console.log(typeof(z)) → Number
```

```
var x = 100
var y = "Pooja"
var z = x + y
console.log(z) → 100Pooja
console.log(typeof(z)) String
```

```
var x = 100;
var y = true;
var z = x + y;
console.log(z); → 101
console.log(typeof(z)); Number
```

```
var x = 100
var y = null
var z = x + y
console.log(z) → 100
console.log(typeof(z)) Number
```

```
var x = 55
var y = undefined
var z = x + y
console.log(z) - NaN
console.log(typeof(z)); Number
```

```
x = undefined
y = undefined
z = x + y
console.log(z) → NaN
console.log(typeof(z)) → Number
```

* Hypertext:- A text representation contains references of other text.
ie one webpage contain reference to another webpage.

* Tag:- instruction to browser

Hypertext is text, that can produce documents (web pages), that can contain reference to other documents (web pages).

Markup language → instruction in form of tag
→ syntax in the form of tag.

Html can be executed by browser.

* Browser:- software that can read, understand, execute html & display output (web page) to the user.

Html was developed by "Tim Berners Lee" - http protocol
↳ W3C world wide web consortium

Browser can execute only 3 lang Html, CSS, JS.
client side

* Tag :- Command / instruction to browser to display some specific o/p.

Html program can written in upper / lower case but lower case recommended.

* Types of tags:-

① Paired tags

<tag> </tag>

② Unpaired tags:-

<tag> or <tag />

JavaScript:- Prototype Object Oriented lang

* Syntax of html:-

<html> - starting pt of html program.

<head> } non content info - info that is part
</head> } of web page but u don't want to display

<body> } content - info that u want to display in
</body> } web page.

</html> → ending pt of html program

html can work on any code editor. recommended - VScode

`<title> Welcome </title>` - represents title of web page that displays titlebar of browser

Html doesn't support compilation.

* Basic tags of html:-

1) Heading:- to create headings in web page
 - supports 6 levels of headings.
 - all heading tags are paired tag
 ex h1 to h6

2) Paragraphs:- Few lines of text.

- `<p>` tag is used to create paragraphs
 - paired tag. `<p>` `</p>`

head-body → sibling relation.

- A paragraph can't contain another paragraph.

* Text formatting tags:-

`<b>` `</b>` - bold

`<p>` You need help from `<b>` Pooja `</b>` always
 → here Pooja will be in bold.

`<i>` `</i>` - italic.

`<u>` `</u>` - underline.

`<strike>` `</strike>` - strikethrough.

`<sup>` `</sup>` - superscript.

`<sub>` `</sub>` - subscript.

Pooja

Pooja

~~Pooja~~

® Pooja

- upside of line

Pooja. - downside of line

* images:-

- to display in web page, we require jpg or png

JPG - joint photograph Graphic

PNG - Portable Network Graphic

 - used to display image.

→ unpaired tag. no closing.

↳ src - source

src is attribute of 'img' tag

Attribute is "detail" about particular tag

* Attributes of 'img' tag:-

1. src :- specifies source path of image file.

"filename.extension"

It is recommended to copy the file into current folder in which html file is present

2. alt :- browser unable to get the image, alt text appears hence alt is imp.

if image appear alt text appear

if alt text appear image do not appear.

3. width - horizontal size

4. height: vertical size

* Hyperlinks:-

- are used to create links (references) from one web page to another web page.

<a> tag - anchor tag used to create hyperlink

href - attribute - hyperlink reference

 link text here

 Click here

 Google

List tags:-

used to display list of names in web page

Types of list tags:-

Un-ordered list `<ul>` - contains any no. of `<li>` tags.

Ordered list `<ol>`

Un-ordered list

- item 1
- item 2
- item 3.

`<ul>`

`<li> item 1 </li>`

`<li> item 2 </li>`

`</ul>`

Ordered list

1. item 1
2. item 2
3. item 3.

`<ol>`

`<li> item 1 </li>`

`<li> item 2 </li>`

`</ol>`

if u want to change bullet symbol use CSS, use option

HTML → used to create elements in web page.

CSS → used to apply styles to element.

JavaScript → used to create functionality.

* Table Tags:-

- used to display tables in web page.

`<table>` → represent entire table

`<caption>` → table title

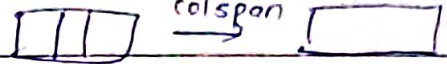
`<tr>` → table row

`<th>` → header cell in table (table header)

`<td>` → normal cell (table data)

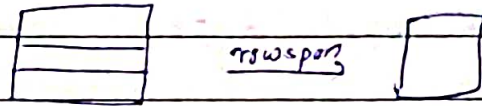
1) colspan:-

Merging the cells horizontally



2) rowspan:-

Merging the cells vertically



* HTML forms:-

Form is collection "form elements" such as checkbox, radio button, dropdownlist etc.

- used to collect details from user

<form> <label> <option> <legend>
<input> <button> <optgroup> <datalist>
<textarea> <select> <fieldset>

<form>

all remaining tags of forms:
 - next line

</form>

form-specifics starting & ending points of form

① input:-

<input> tag used to create various types of form elements.

eg. textbox, checkbox.

unpaired tag.

<input> tag must be used only inside <form> tag.

<input type="text">

tag attribute value of attribute.

* Attributes of input tag:-

- | | | |
|---------------|------------|-------------|
| ① type | ② checked | ⑩ autofocus |
| ② maxlength | ⑦ disabled | ⑪ required |
| ③ value | ⑧ min | |
| ④ name | ⑨ max | |
| ⑤ placeholder | | |

type of type:- text, password, url, submit, email

`<input type = "text">`

`<input type = "password">`

Check boxes:- 2 options only yes or no.
more than 2 options - multiple $\Rightarrow$ radio button

☒ checked state : yes

☐ unchecked state : no.

`<input type = "checkbox">`

Radio buttons:- Group nature
used to display set of options to user & allow the user to select any one of them.

`<input type = "radio">`

`<input type = "radio" name = "course">` Jan

`<input type = "radio" name = "course">` Hm

`<input type = "radio" name = "course">` rmt

can select any one of the multiple.

Check box nature $\rightarrow$ individual

Radio button nature $\rightarrow$ collective, group nature

Auto complete:-

auto complete on, browser maintains history of all values entered into textbox & will show later in textbox that has same name

* JavaScript:-

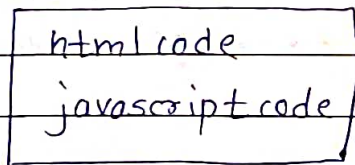
- programming language, used to create functionality in web page
- reading i/p, processing & providing o/p.
ex. displaying menu when user clicks menu icons.
- provides programming concepts such as variables, data types, operators, arrays.

- case sensitive language.

- OOP → developed by Netscape Corporation in 1996.

1. Internal JS

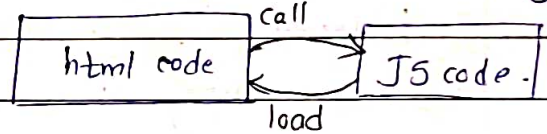
filename.html



2. External JS

filename.html

filename.js



both html & JS code written within same file.

- dis → code length ↑ create confⁿ
- suitable for practice learnⁿ

JS file will loaded into html

- even length ↑, easy to understand

- No. of files ↑

- reusability of 1 js file in multiple html files.

1. placing `<script>` in `<head>` tag.

Dis →

User will not see o/p until execution of both html & js code is finished.

1. placing `<script>` in `<body>` tag.

Adv.

User will see o/p. immediately after completion of execuⁿ of html code in `<body>`.

filename.html

`<head>`

`<script>`

js code

`</script>`

`</head>`

`<body>`

content here

`<script>`

js code

`</script>`

`</body>`

* Output statement:-

- to display some c/p value to user.
- "console.log()" is c/p statement in JS
- console.log(value);
- to see c/p in "browser console".

* Variables:-

We can change datatype of variable in JS

var variable name:-

- var - keyword, used to declare variable. When variable is declared, memory will not be allocated. When we assign value, memory will be allocated.
- variable name cannot be same as keyword.

$a += b \Rightarrow a = a + b$ $a++$

$a -= b \Rightarrow a = a - b$ $a--$

$a *= b \Rightarrow a = a * b$

$a /= b \Rightarrow a = a / b$

* Concatenation operator (+) =

num + num $\Rightarrow$ addition.

any thing in double quote $\rightarrow$ String + num $\Rightarrow$ concatenation

num + string $\Rightarrow$ concatenation

String + string $\Rightarrow$ concatenation.

* Control Statement:-

By default all statements executes in top-to-bottom i.e. sequential order

1) Conditional $\rightarrow$ if else
switch case.

2) Looping $\rightarrow$ while
do while
for

loops $\rightarrow$ for, while, do while.

* Functions:-

Function is a collⁿ of statements, to perform specific operⁿ

Day 6:

* Document Object Model (DOM):-

selectors in JS - finding by ID, Name, etc.

`<input type="text" required="required" title="Enter ur name">`

title attribute (default title msg) When we bring mouse near our empty box please fill required field (due to required) to change this msg on tool tip define `type = Please enter name.`

`<input type="radio" name="gender"> M`

`<input type="radio" name="gender"> F`

when we give name it will select any one of both gender i.e. name=gender forms a group

→ name attribute is used to store as key-value pair name-key value entered-value.

→ but name attribute can't be used for finding! so we use id attribute to search uniquely ensure that id is unique. becoz it's not unique.
↓
kind of name used to search.

Content Type - program.

html → html parser Browser → ① read &

js → js engine ~~browser~~

xml → xml parser.

② create objects specific to UI

JS can't understand tags but it can understand objects. Hence objects are created by browser.

Page No.

Date

Step ② creation of objects

01(html) 01 = new HTMLElement("html") → <html>
02(head) 02 = new HTMLElement("head") → <head>
03(title) 03 = new HTMLElement("title") → <title> Demo </title>
03.innerText = "Demo" </head>
04(body) <body>
For every tag, corresponding obj. is created in memory in hierarchical manner in which
05(input) in hierarchical manner in which
• type = "text" html is written.
• title = "Enter username".

These objects are for scripting. Objects made in html are available in script (JavaScript engine).
JavaScript communicate with these objects not with html.

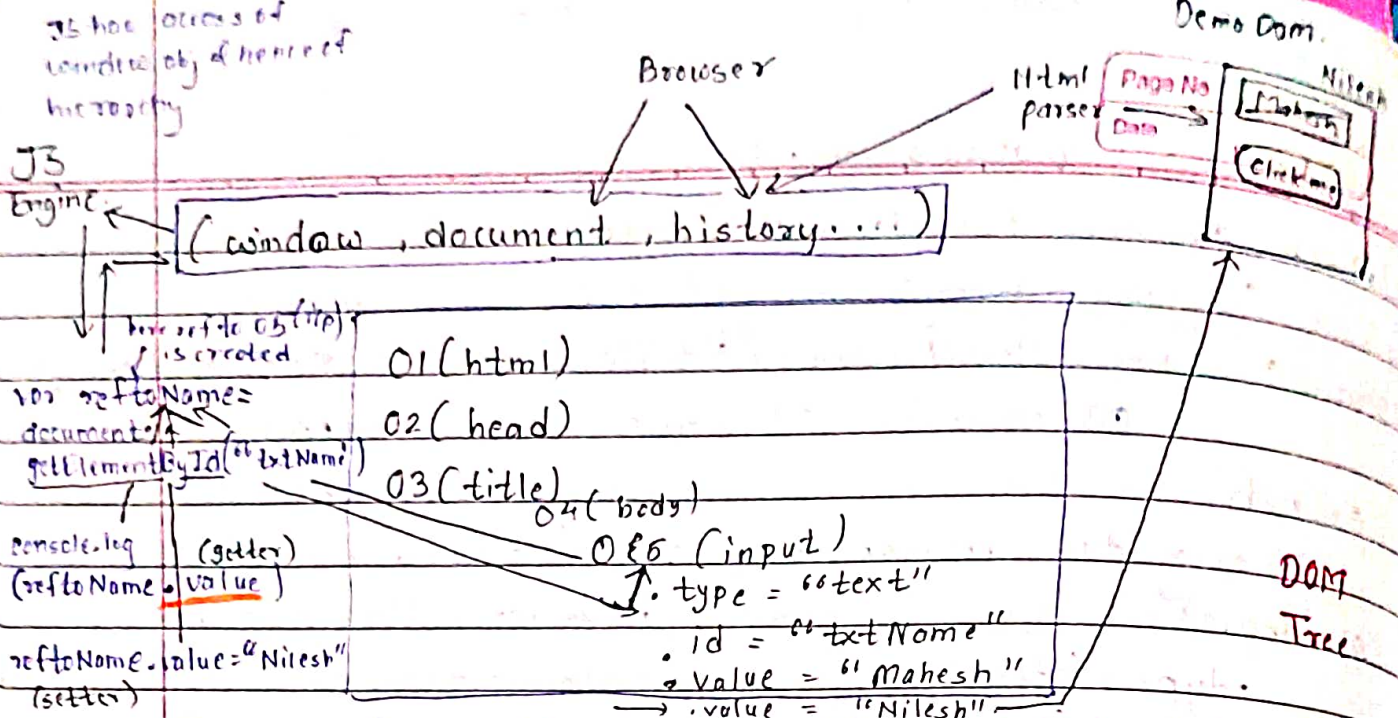
I.Q Interview of JS is on DOM only.

Browser:- is the controller betⁿ JS engine & html parser. It communicates betⁿ two. html parser makes obj. & JS engine talk with that obj not with parser directly.

For each tag → object
per Attribute → getter, setter
ex. setInterval, clearInterval
this obj. comes up with some helper functions.

→ When we click open a browser, 1 window obj. is created
window only one obj. is created. & it has many objects
ex. document, History, Navigator

after window when html page is called, their another objects are created. document & history obj. have access of all these objects created due to html program.



Browser have continue watch If any value is changed inside object it is reflected on UI, UI value is also changed when there is chge in object hierarchy

Js engine never talk with html parser hence browser comes in betⁿ as mediator.

Whenever u have js code dependent on html input control ex. input tag. we write js code at end in body if written in head, object of ip in body is not created at that pt. so u can't access elements & code not run properly.

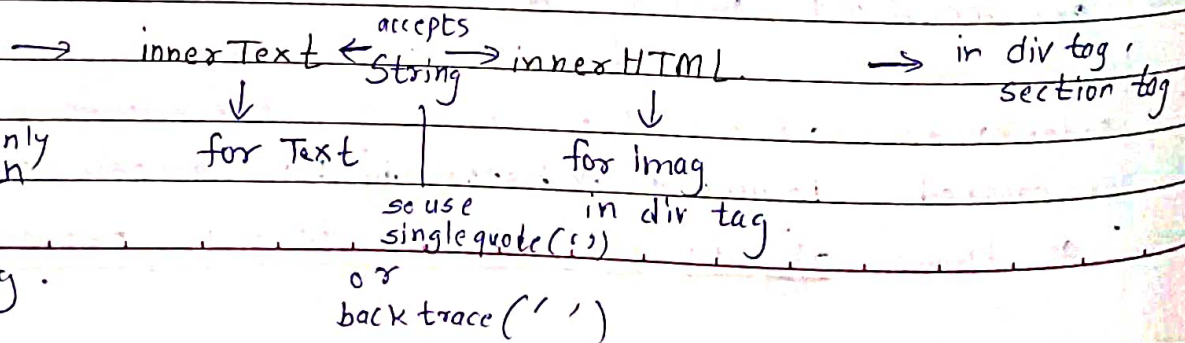
In this case. html code will run but js will have problem (if js written in head).

Attribute of obj can changed in 2 ways:-

Test Dom2

- ① from UI it will change in code
- ② change in code it will update on UI.

these 2 properly will work only with paired tag.



F11 - navigate inside the "fu" which is called.
 F10 - debug line by line.

```
<script src = "validation.js" > </script>
```

fu written in separate js file.

```
<script>
```

```
function Execute () { ?
```

```
</script>
```

we can write any no. of script tag.

→ here we give call to html file & html will give call to js (when page & js page will be downloaded just as in case of .css file). in network window we can see this. ^(when script is encountered)

validation.min.js → minimized version of validation.js

webpack → packs the code, minimize version of js.

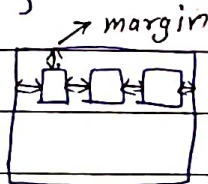
minifier.org

teptal → changes the code, variables, better operators

TestDOM Manipul'01 → Div on click color chge

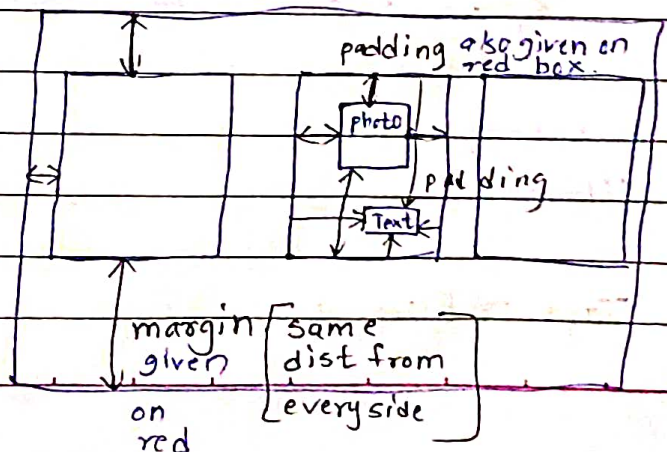
TestDOM Manipul'02 → Div on tip of mouse

Margin :- many no. of element with same dist.



margin is distance outside our current box.

padding is distance inside our current box.



By default alignment is from left

2 values → margin 10px 20px
 ↳ top ↳ bottom left right
 bottom.

check this in lab.

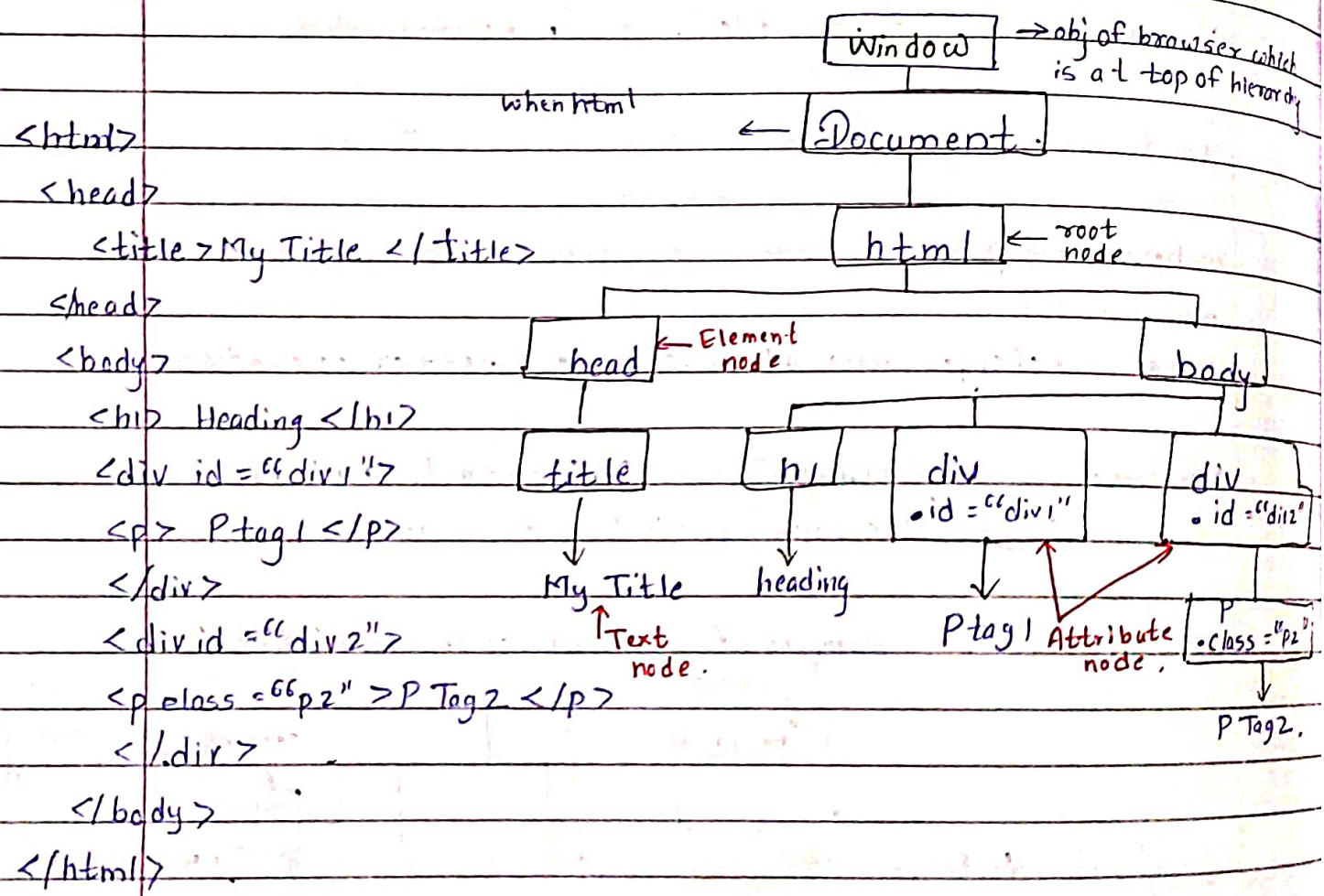
3 values ① top ② right ③ left ④ bottom
 1st value 2nd value 3rd value.

Browser

margin / border.

4 values top right bottom left (clockwise).

^{engine}
Javascript is single threaded. but browser is multi-threaded.
 It can't execute multiple statement at a time.
 it execute one statement at a time.



Tags ~ Element

//root node

`console.log(document.documentElement);` → `html`

`console.log(document.body)` → body

console.log (document.head) → head

console.log(document.body.children) → h1, div, div.

Html Collection (3) [h1, div, div]

- value [Input Text value Property]

→ value property sets or returns the value of value attribute of text field.

→ value property contains the default value or the value a user types

`textobject.value` → return the value property

`textobject.value = text` → set the value property.

Day 7

• XML - Extensible Markup Language

HTML parser along with browser creates object

Executⁿ of window obj $\rightarrow$ browser's job

→ window. setInterval (Increase, 1000);
like funⁿ ptr. ↳ time lapse.

Browser thread ~~to~~ tells to JS to call increase funⁿ after interval of 1000ms. hence it seems js is multithreaded but it's not true. browser in background is cooking.

* Alex:-

When alert is called it freezes background page until we click OK button.

When we click alert button, counting stopped & popup displayed
when we click OK, counting resumed.

In this case increase fuⁿ is called by each queue hence no. are printed in less time.
each time we click, new queue is formed

Page No.

Date

Once we click, it ↑ the counter after given interval is here. but when we go on clicking that many loops are formed & counter is ↑ faster. it will not take time to go next counter now it take < 1s.

browser → returns a token no to js engine (token no. can be any random no. which is no. of callback queue)

`window.setTimeout(Increase, 1000);`

Call a fuⁿ after interval only once → setTimeout

Call a fuⁿ after interval again & again → setInterval

Client side web API:- fuⁿ of browser which are reusable
ex. `setTimeout`, `setInterval`, `clearInterval`.

Script code

`console.log("Hi");`

Token no.

`console.log("bye")`

Call Stack.

JS thread

JS Engine

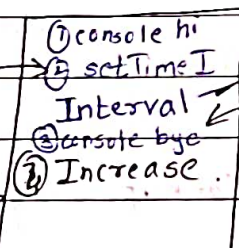
Browser APIs

Execution Area

browser thread →

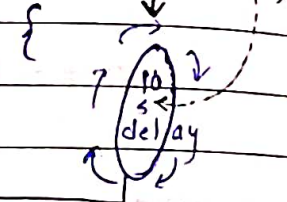
`window.setTimeout(Increase, 1000)`

token no.



event loop (Infinite loop)
checks in queue if there is any fuⁿ to execute & also checks event is happening (e.g. click)

callback queue.



Console → "Hi" token No "bye" counter printed here

Every fuⁿ in js returns something hence when `window.setTimeout` is given call (to browser) js engine wait for reply & browser give token no. in reply.

After this browser thread will execute `setTimeout` fuⁿ & delay of 10s will take place.

Till then after token no. js think `setTimeout` is over & go to next fuⁿ i.e. `bye` & js is idle now

After 10s browser will call increase fun which is fun of js. Hence to go to js increase is enqueued in a ^{callback} queue for js engine. Js engine will execute the fun in queue.

Js engine is in infinite event loop it keeps on checking work for it in callback queue or an event is happening (click).

event loop as in it's in a wait, waiting for event/queue

Token no → no related to particular callback queue

To remove queue:-

window.clearTimeout(Token No)

Fun is called after 10s only if condⁿ are idle. If there is another fun which is interrupting timeout fun, may be timeout will take delay of more than 10s.

* Array in Js:-

→ with each click new queue is formed. We can store each queue's token no. in an Array.

```
var allTokenNo = new Array();
```

↳ Growable, variable size.

```
allTokenNo.push('Token No');
```

no need to specify size when we declare array.

allTokenNo.length → size of array

```
for (
```

```
    window.clearInterval(allTokenNo[i]);
```

Array in Js can work as stack, push, pop. → close all queue at a time.

↳ Last In first out.

```
var queueToClose = allTokenNo.pop();
```

```
    window.clearInterval(queueToClose)
```

→ close the queue one by one.

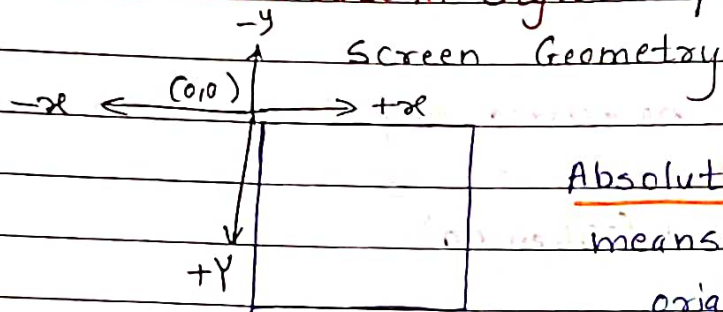
* Disabled attribute:-

Disables the button. disabled = "disabled"

To enable disabled button, assign null to that button

ref.disabled = null → to enable the button

* Absolute Attribute in Style:- position - absolute



Absolute position
means start from
origin (0,0).

* Opacity:-

1 → full fledged 100% on screen. 0 to 1
0 → full opaque - no op on screen. opacity changes.

* Flex → div display attribute:-

arrange & wrap the content using flex.

<div style = "display : flex ;"

* `\${}` - template literal Syntax:-

→ for string formation.

var people = [

array of object { No: , Name: , City: },

json obj → { No: , Name: , City: },

{ No: , Name: , City: }

]

regres.in → Test ur front-end against a real API
regres.in/apilusers → fake data.

Page No.

Date

Server side job:- → Fires Insert, Update, Delete queries
make the array & sends data to browser.

Browser side job:- browser side person creates an array.
accepts the array & iterate the loop & gets/prints data.

frontend → react, js, jquery,

JSON → Javascript Object Notation

website using JSON was faster than XML.

JSON was giving the splitted form of string.

At start, there was very less support of server
side. i.e. java, net PHP was not supporting JSON.

In 2008, android came at that time JSON was
faster than XML was observed so server side
technology started supporting json.

so nowadays JSON is used all over becaz less data
is transferred.

2 predefined obj in browser XML, JSON.

Javascript.

Java .net

PHP

Ruby

obj → json

XML - eXtensible Markup Language

```
<?xml?>
<person>
  <no>1</no>
</person>
```

JSON.org

(json.js)

json to obj

```
{no:1, name: "mahesh", city: "pune"}
```

Java Script Object

Notation (JSON)

After 2008, Java, net
started supporting
json

* JSON:-

- syntax for storing and exchanging data. It is a text written with Javascript object notation.

XML software parsing process can take long time. JSON uses less data overall, so u reduce the cost & increase the parsing speed.

JSON

quick process

- fetch a JSON string
- JSON.parse the json string.
- more human readable
- supports array

XML

lengthy process

- fetch a XML document
- use XML DOM to loop through doc.
- extract values & store in variables.
- not support array.

JS object can be used as JSON.

when client wants to see data only in form of text JSON is used.

JSON.parse → converts JSON to string

- helps to transfer data from server.
- transmit data betⁿ server & web application using JSON
- JSON - language independent
- curly braces → object
- Square braces → array.

```
{ "name": "John", "age": 30 }
```


Day 8.

SEO - Search engine optimisation

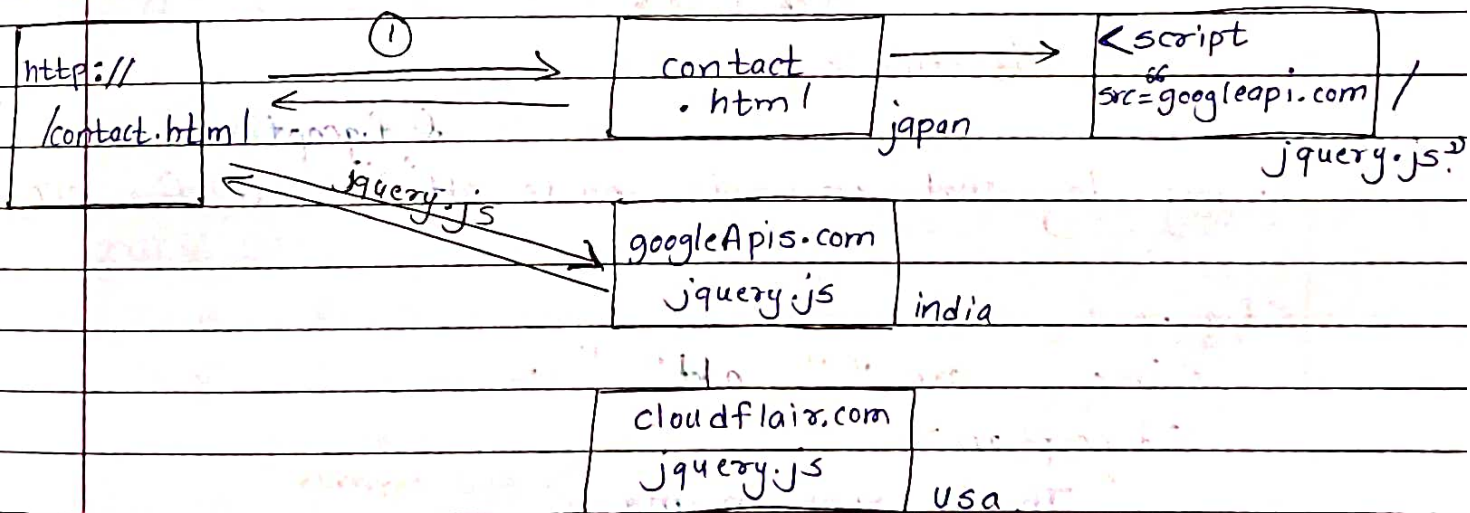
- HTML 5 controls - SEO friendly
- Diff. input controls like Radio, Check Box, Select-Options
- basic validations
- HTML 5 API - Local storage Vs. Session storage.
- HTML 5 API - Canvas, Video, Audio

Proxy Server

- we can use proxy websites for browsing only but can't use for private credentials. it's like we're submitting our password to proxy websites

CDN - Content Delivery Network

Client Browser



Why google will do this free?

To monitor network traffic. Google bot reads the query search by client's entered - Sunbeam info. google bot stores that call is given to sunbeam.

Html 5 controls / Tags - SEO friendly

Header, Footer, Section, Article, Figure, Figcaption

Aside, Nav → used in bootstrap for menu bar
SEO opt.

Aside:- If there is any type of add of other site in ur site, keep that in aside to ignore SEO of ad.

This content will be hidden for search engine optimization. ex. remember sir's ex. ^{best} order samosa search kiya chahiye

Article:-

instead of paragraph

<section>

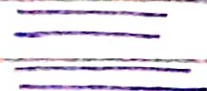
<h1> Sunbeam </h1>

<Article>

</Article>

</section>

Sunbeam



& figcaption

In img tag what we write inside alt is used for SEO

<figure>

<figcaption>

This is sunbeam img

</figcaption>

SEO Keywords

* Input controls - radio, checkbox

1] Radio:- To group all radio button give common name to all button

getElementsByName → gets array of radio buttons

Test input
controls.html

refToGender[0].checked → true → To check which
refToGender[1].checked → false button is selected

* <select>

like input, dropdown.

country: <select id=" " ">

<option value="91">India</option>

<option value="1">USA</option>

</select>

but this selects India by default
& then u can change.

multiple="multiple" → allows user to select multiple
value from dropdown.

→ refToHobby.children[0].selected → True/False.
refToHobby.children[0].value → 11.

Onchange, Onclick, Onselect

or we can have:
 .NET

 PGDAC

 PGDBDA

 .NET

 java.

 DITISS

• PGDAC

• PGDBDA

• .NET

• Java

• DITISS.

Menu are created using bootstrap.css.

* Canvas:-

Graph, freehand

- replaces need of plugin - used previously but not secured
- lookless containers, used for drawing purpose

we can't use multiple version of IE. but can use recent updated IE

If our browser not support HTML5 tags then canvas, video, audio are treated as Malicious tag useless. it will print whatever content it has.

* getContext:-

reference to both axes (x & y) i.e. dimensions

`refToMyCanvas.getContext("2d");`

viewport 3D → 3D drawing in canvas

- `moveTo(10,10)` → start from 10,10 pt
- `lineTo(50,50)` → another pt., draw till this
- `stroke()` → draws line bet. 2 pts. $(10,10)$ to $(50,50)$

`overflow=scroll` → scrollbar.

~~gag~~ `google`

`google money` → go right click & inspect

* Video:-

`<video src= id="my video controls="controls">`
`</video>`

↳ display all fun
like play speed change

`ref.play()` → to play video on click.

autoplay = "auto"; → plays video automatically, no need of play

Day 9.

LocalStorage vs SessionStorage.

ES6 →

Arrow Function (s) : Map

Arrow Function

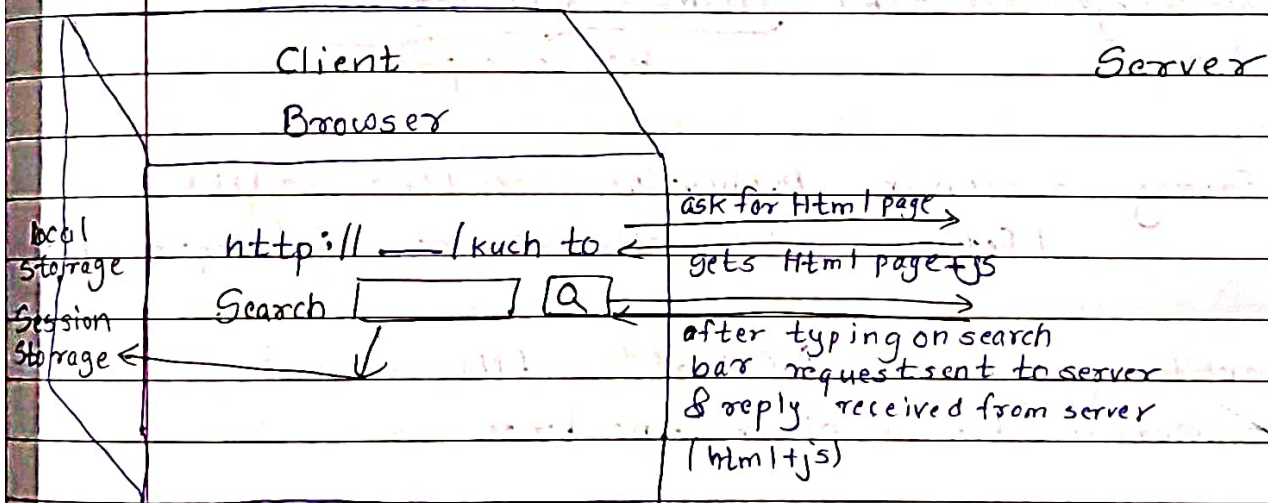
Template literal $\{ \{ \text{variable} \}$

Class keyword syntax.

Let & const.

XmlHttpRequest Object Usage.

JQuery Basics → not used now much more.



When we login first time, browser gets a token & server also stores a token. After some time if u want to login again server just checks token & u don't need to login again. That token is stored in RAM. If u click remember me then token is stored on harddisk / secondary memory.

* ES 6:-

1] Array function - Map → similar to for each fuⁿ Arrays.forEach
Arrays.Map.

```
for (var i=0; i < days.length; i++)  
{  
    Print ( days[i] );  
}
```

instead of iterating use Map fuⁿ

increases
readability

→ days.Map(Print) or Map(days, Print)

this is similar to

```
function Map ( days, Print )  
{  
    ↪ fun is passed as parameter.
```

```
for ( var i=0; i < days.length(); i++)  
{  
    Print ( days[i] );  
}
```

days.Map(Print) → here days ^{passed} ~~gets~~ ^{is} a first parameter
& Print is second parameter
array on which map fuⁿ is called is passed as 1st para.

→ Map fuⁿ is written just to give flavor of java

→ Inheritance in case of constructor function:-

```
Employee.prototype = Person(); - ES5
```

called as prototypical

→ Inheritance - class writing using class - ES6. extends ^{↓ inheritance} concept
keyword is used

→ Class using Literal Syntax.

```
Employee.== proto == Person.
```

Window obj has key value pair

& some key have value as a -fu?

$\text{setInterval} = \underbrace{f}_{\text{value but fn}}$

Even if u ^{don't} give proto, by default proto is constructor

Employee. - proto - $\rightarrow$ give data of person

Employee: --proto-- → give data of person
Employee: --proto-- --proto-- } → parent of person
or Person: --proto-- } → constructor

or Person. -- proto --

Employee. -- proto -- . -- proto -- . -- proto -- } → parent of other
Person. -- proto -- . -- proto } → null

Person, -- prete -- , -- prete

--proto is so confusing so in ES5 prototype word is used.

* Arrow function :-

→ var Add = (x, y) $\Rightarrow$ { return x+y; }

in js where ever u write () for fu², fu² is called
at that place

```
setInterval(Print, 1000)
```

↳ this passed as parameter
we don't want it to be executed here
we have only written name not ()

→ `setInterval()` →

```

} console.log("H.");
y = 1000);

```


To change colour of line → `strokestyle`

Page No.

Date

* let:- in ES6

define the scope using `let`

local scope content

`let` can be replace with `var` but `var` can't replaced with `let`.

if `let` is defined inside `if`, it has scope only inside `if`.

when ES6 is converted to ES5, `let` is converted to `var`.

* Const:-

`const Add = function(x, y) { return x+y; };`

`Add = "abcd";` → error Assignment to const. variable.

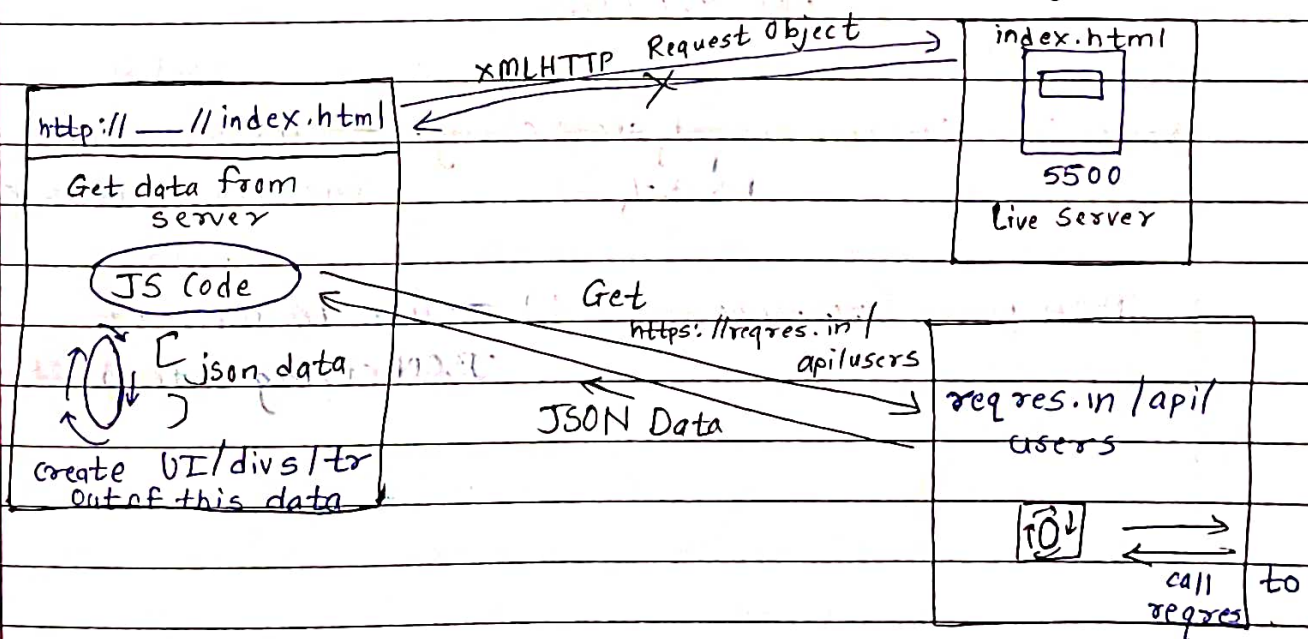
`console.log(Add(10, 20));`

But in ES5 ultimately `const` becomes `var`.

Submit → Form tag req., forms key value pair:

Button → click.

* Diff betⁿ `input type="button"` & `button` tag.



Wherever u see dynamic content on screen, u are shooting frequently to server at some interval. For that use setInterval.

TheMealDB → free API, Rapid API, Rapido.

Page No.

Date

action & method, get post. default req is GET (ie. in header).
data will be visible. ← post - data dekhna padega

- ① Helper create `var helper = new XMLHttpRequest();`
- ② Packet created `helper.open("GET", "https://reqres.in/api");`
- ③ Packet sent `helper.send();`

```
var helper = new XMLHttpRequest();
```

// helper.readyState = 0 → start point.

```
helper.open("GET", "https://reqres.in/api/users");
```

// helper.readyState = 1 → packet created.

```
helper.send();
```

// helper.readyState = 2 → writing for reply packet sent.

3 → waiting for reply

`helperToCallServer.status = 200` ⇒ 4 → reply received

else 404-error

`helperToCallServer.onreadystatechange` → this is called

1-2-3-4

Whenever state is changed

`JSON.parse(response received)` ⇒ use to convert
JSON string into JSON obj

Day 10

→ Why JQuery?

api.jquery.com

1 → solution to common cross browser issues
 jquery.js → includes some common soln to cross browser issue

2 → Selectors → always gives array

3 → Support for JSON. = fun in jquery which i/p JSON & also o/p JSON

production version - minify file / minimized version.

development version - normal file in which code is written.

we can write .js file anywhere in code becoz it's has no dependency in program it's a generic code.

```
jQuery("#txtName")
```

↳ # tells that txtName is id

```
var ToTxtName = document.getElementById("txtName");  
console.log(refToTxtName.value);
```

```
or  
console.log(jQuery("#txtName").val());
```

```
<script src="jquery-3.6.1.min.js"></script>
```

↳ import jquery.

on clicks = "Do something()"; → static call to fun on click.

Dynamic call to fuⁿ programmatically.

① ~~var~~ var refToBtn = document.getElementById("btn");
refToBtn.addEventListener("click", Dosomething);

↳ fuⁿ written here
becoz fuⁿ need here () not written
not to be called here.

② jQuery("#btn").click(Dosomething);
↳ ref. to button by getElementById
↳ fuⁿ call

find ref. to button & then on click call Dosomething

instead of jQuery & \$

③ \$("#btn").click(function() {
console.log("Hello" + \$("#txtName").val());

further remove fuⁿ & use arrow fuⁿ.

→ for div:- when there is single div:-

selectors:-

i) \$("#btn").click(function() {
\$("#myDiv1").css("background-color", "red");
\$("#myDiv1").css("width", "200px");
} instead of style use .css directly

ii) \$("#myDiv1").css("height", "200px")
.css("width", "200px");

instead of finding myDiv twice/thrice write
css only anytime a req.

Toggle fun in jquery - If B is applied apply A & vice versa

Page No.

Date

iii) further improvement

key value pairs of json obj

```
$("#myDiv1").css({ "background-color": "red",  
                    "height": "200px",  
                    "width": "200px" });
```

→ when there are multiple divs in a code :-

\$("div") → find by tag → getElement by Tag Name

\$(".myStyle") → find by class → getElement by class Name

\$("div.myStyle") → div who has class name is myStyle
tag name class ① ②

write acc. to hierarchy

or

\$("div").filter(".myStyle").css → will first find
div tag then filter out
by class myStyle

\$("div[title = 'Panji']").css → find div with Panji

wildcards

[1] → \$("div[title ^ = 'P']").css → title starting with P

[\$] → \$("div[title \$ = 'i']").css → title ending with i.

[*] → \$("div[title * = 'n']").css → title containing n.

else:

\$("div").filter(".myStyle").filter("[title ^ = 'P']").css =
u can write chain of filters

```

$( "tr" ). mouseover ( function () {
    $( this ). css ( "background color", "orange" );
    ↳ to select current tr.
}

```

```

$( "tr" ). mouseout ( function () {
    $( this ). css
}

```

```

$( "tr" ). click ( function () {
    ↓
    $( this ). css ( "back color", "green" );
    $( this ). unbind ( "mouseover mouseout" );
    ↳ undo the mouseout change.
} )

```

Use jquery logically. like don't use jquery for only writing ^{Hi program} becoz it downloads file of 88KB of jquery. [Hi program will be of half line only but 88kb will be downloaded. It's beneficial if ur no. of lines are more to become or to make it consize.

\$. ajax → global function
→ case sensitive & space sensitive

```

$. ajax ( {
    url : " ",
    method : "Get" / "Post",
    contentType : "application / json",
    success : function ( result ) {
        debugger;
    },
    error : function () {
        debugger;
    }
} );

```

no need of
parse here


```
success : function (result) {
    var people = result.data;
```

```
    people.map(function (person) {
        $( "#myTable" ).append(
            <tr>
                <td>
                    ${person.id}
                </td>
            </tr>
        ) } );
```

Annotations:
 - *instead of innerText = innerText + optional* (pointing to the append method)
 - *getElementBy ID* (pointing to the `#myTable` selector)
 - *Tag* (pointing to the `<tr>` tag)
 - *for ID* (pointing to the `id` attribute in the `<td>` tag)

Add & show on screen → TestMisc2.html.

remove or removechild → To remove the object

find obj programmatically instead of finding ctrl on UI

Day 11

Ajax - Asynchronous Javascript and XML

Term given by google in 2003 JSON introduced in 2008
 hence XML is there in Ajax.

JSON.stringify:- JSON in string
converts JSON obj into string.

JSON.parse - String into JSON object.

OnClick:- will execute a fun only after clicking the button. when we click fun is called.

addEventListener - Programmatical way of giving instⁿ on click
 if (onclick select user1)

else if (onclick select user2)

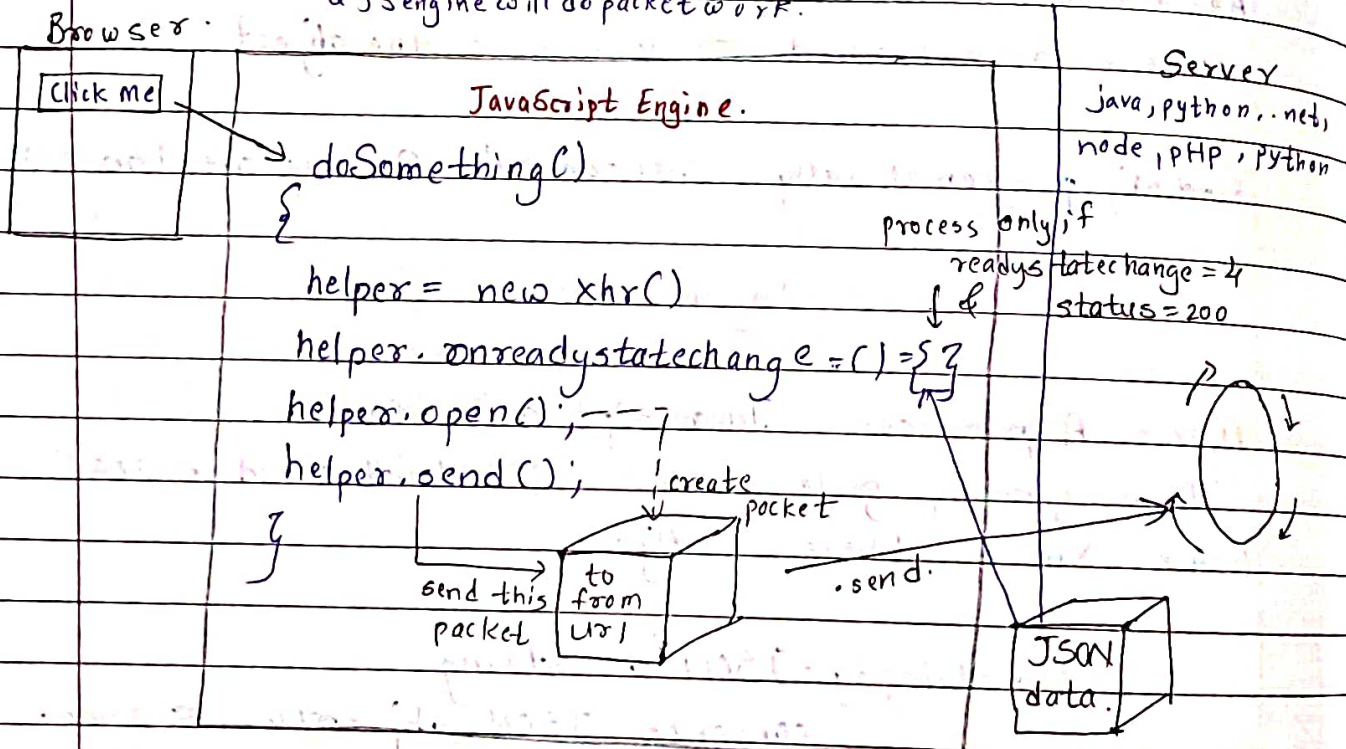
else if (onclick if null → nothing)

in case of if null → even if a click nothing will happen

In React, we will call data from ~~www~~ server in form of JSON, using fetch()

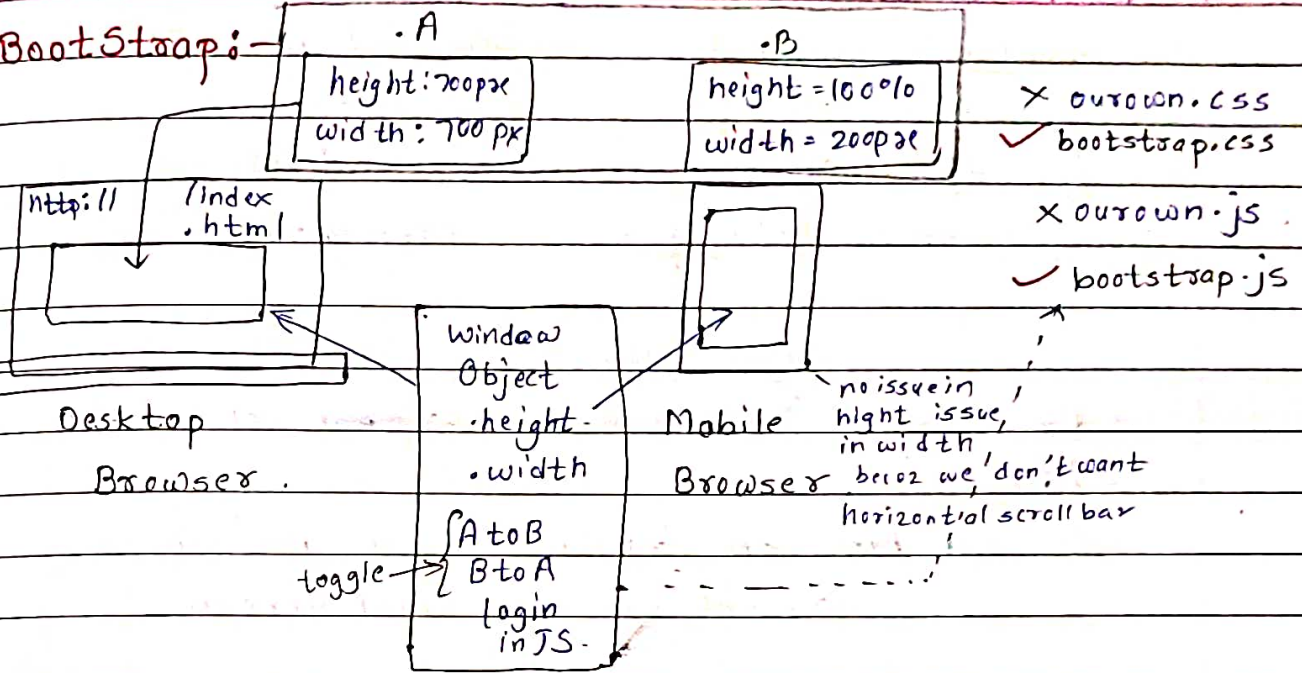
- Methods to send data to server:- URL - request response in ? name
- ① in URL send data using ? ex. paneer
 - ② `helper.send()` → If we want to send data to server, we'll give parameter inside send e.g. `send(string)`, `send(obj)`

when we click browser will call JS engine & JS engine will do packet work.



when we click, browser calls JS Engine & do job of packet (open, send) etc.

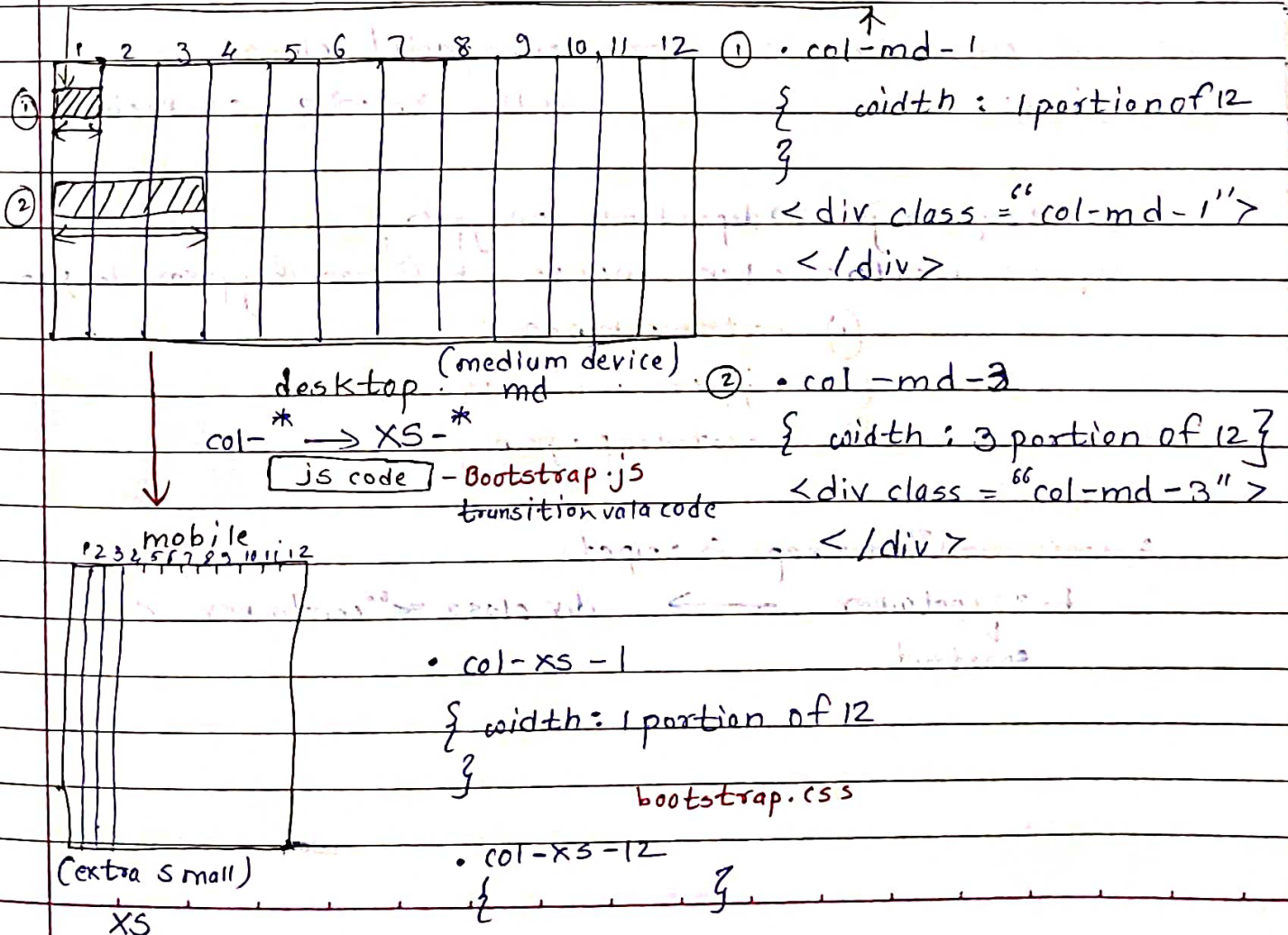
* Bootstrap:-



JQuery

→ 12 common Grid theory:-

screen is logically divided in 12 equal parts.



For desktop :- col-md-1 ... col-md-12

For mobile :- col-xs-1 ... col-xs-12

For tablet :- col-sm-1 ... col-sm-12

Larger screen - col-lg-1 ... col-lg-12

```
<div class = "col-md-1 col-md col-xs-12">
</div>
```

1 block on medium screen will occupy
12 block on extra small screen.

```
<div class = "col-md-4 col-xs-12">
</div>
```

4 block on medium will occupy 12 on extra small
something like flex VI.

→

```
<div class = "row">
```

 → row special class

```
</div>
```

 with XS → 12 on mobile

bootstrap.min.js → dependent on jquery
sequence matters when include bootstrap & jquery in program

① bootstrap.min.css

② jquery.min.js

③ bootstrap.min.js

Extensions → Bootstrap 3 Snippets

bs3: container →

```
div class = "container">
```


↓
shortcut.